

UNIVERSIDAD MAYOR DE SAN ANDRES

CARRERA DE INFORMATICA

PROGRAMACION WEB III

INF-133



PRACTICA NRO 2

Estudiante : Ticona Tapia Jhonatan Gerardo

CI:13971678

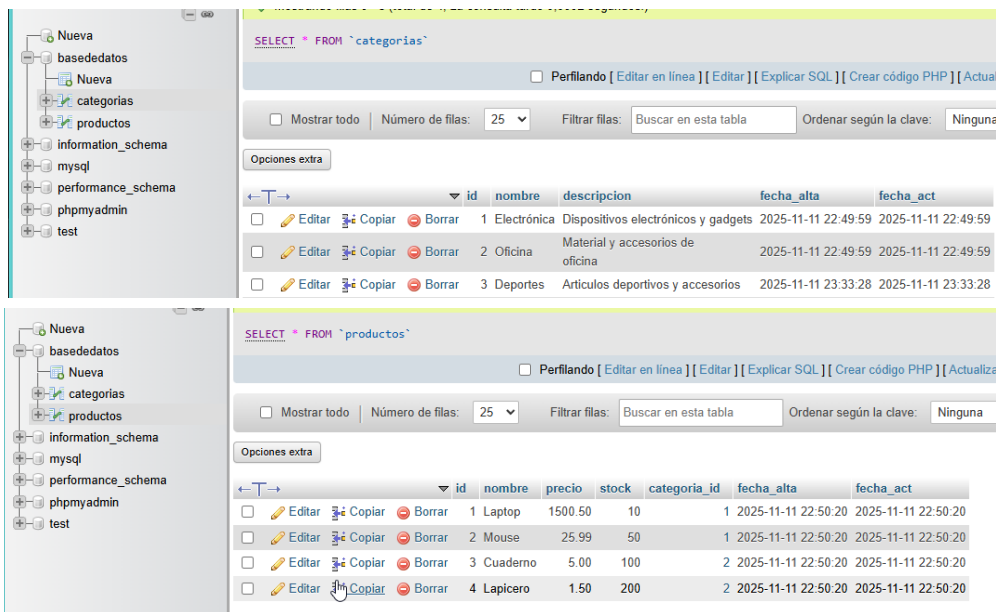
Docente: Lic. Jhonny Felipez Andrade

Fecha: 12/11/2025

LA PAZ - BOLIVIA

2025

Base de Datos Creada:



The image shows two screenshots of a database management interface. The top screenshot displays the 'categorias' table with the following data:

id	nombre	descripcion	fecha_alta	fecha_act
1	Electrónica	Dispositivos electrónicos y gadgets	2025-11-11 22:49:59	2025-11-11 22:49:59
2	Oficina	Material y accesorios de oficina	2025-11-11 22:49:59	2025-11-11 22:49:59
3	Deportes	Artículos deportivos y accesorios	2025-11-11 23:33:28	2025-11-11 23:33:28

The bottom screenshot displays the 'productos' table with the following data:

id	nombre	precio	stock	categoria_id	fecha_alta	fecha_act
1	Laptop	1500.50	10	1	2025-11-11 22:50:20	2025-11-11 22:50:20
2	Mouse	25.99	50	1	2025-11-11 22:50:20	2025-11-11 22:50:20
3	Cuaderno	5.00	100	2	2025-11-11 22:50:20	2025-11-11 22:50:20
4	Lapicero	1.50	200	2	2025-11-11 22:50:20	2025-11-11 22:50:20

Código para los ejercicios:

```
import express from 'express';
import mysql from 'mysql2/promise';

const app = express();
app.use(express.json());

const db = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'basededatos'
});

const puerto = 3001;
app.listen(puerto, () => console.log(`Servidor en http://localhost:${puerto}`));
```

Ejercicio1 POST /categorias que permita registrar una nueva categoría:

Código:

```
app.post('/categorias',
  async (req, res) => {
    const { nombre, descripcion } = req.body;
    const resultado = await db.query(`INSERT INTO categorias4(nombre, descripcion)
    VALUES (?, ?)`, [nombre, descripcion]);
    res.status(201).json(resultado);
  });
```

Resultado:

```
1 {
2   "nombre": "Hogar",
3   "descripcion": "Articulos para el hogar"
4 }

Status: 201 Created Size: 114 Bytes Time: 53 ms

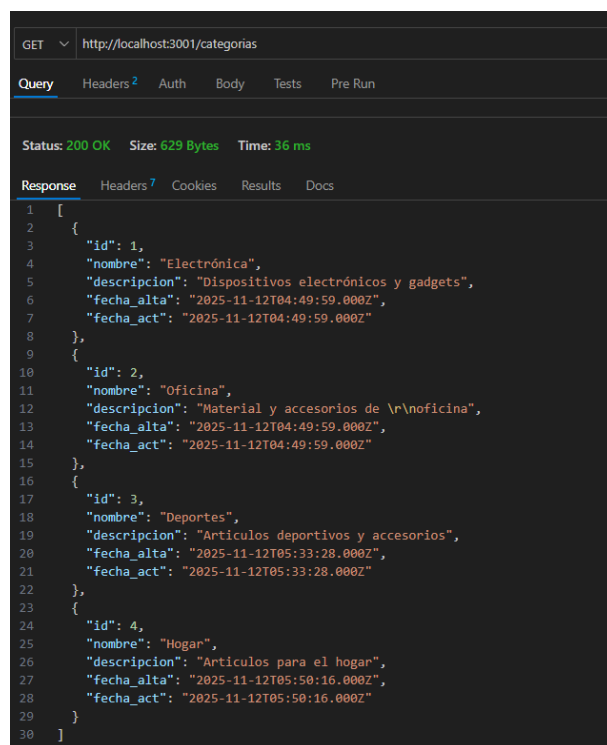
Response Headers Cookies Results Docs
1 [
2   {
3     "fieldCount": 0,
4     "affectedRows": 1,
5     "insertId": 5,
6     "info": "",
7     "serverStatus": 2,
8     "warningStatus": 0,
9     "changedRows": 0
10  },
11  null
12 ]
```

Ejercicio2 GET /categorias que devuelva todas las categorías registradas en la base de datos:

Código:

```
5 app.get('/categorias',
7   async (req, res) => {
8     const [resultado] = await db.query('SELECT * FROM categorias');
9     res.status(200).json(resultado);
10  });
```

Resultado:



GET http://localhost:3001/categorias

Status: 200 OK Size: 629 Bytes Time: 36 ms

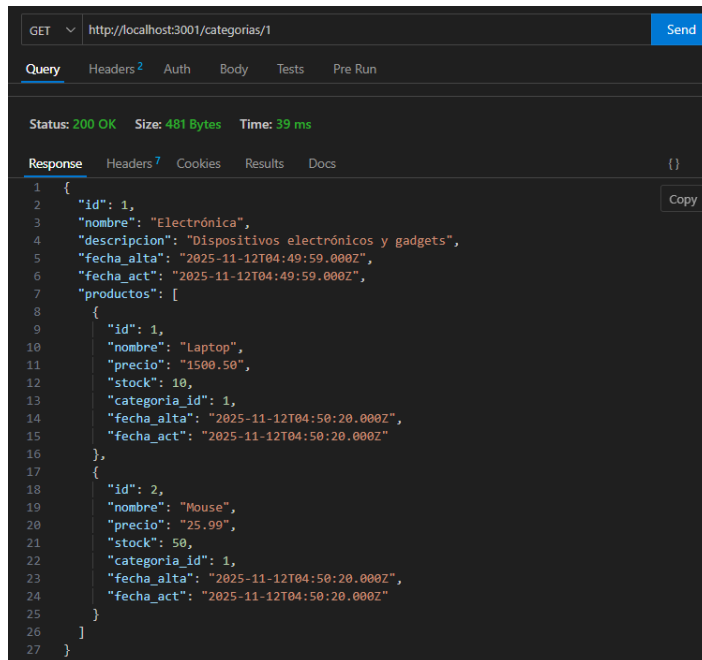
Response

```
1 [
2   {
3     "id": 1,
4     "nombre": "Electrónica",
5     "descripcion": "Dispositivos electrónicos y gadgets",
6     "fecha_alta": "2025-11-12T04:49:59.000Z",
7     "fecha_act": "2025-11-12T04:49:59.000Z"
8   },
9   {
10    "id": 2,
11    "nombre": "Oficina",
12    "descripcion": "Material y accesorios de \r\noficina",
13    "fecha_alta": "2025-11-12T04:49:59.000Z",
14    "fecha_act": "2025-11-12T04:49:59.000Z"
15  },
16  {
17    "id": 3,
18    "nombre": "Deportes",
19    "descripcion": "Articulos deportivos y accesorios",
20    "fecha_alta": "2025-11-12T05:33:28.000Z",
21    "fecha_act": "2025-11-12T05:33:28.000Z"
22  },
23  {
24    "id": 4,
25    "nombre": "Hogar",
26    "descripcion": "Articulos para el hogar",
27    "fecha_alta": "2025-11-12T05:50:16.000Z",
28    "fecha_act": "2025-11-12T05:50:16.000Z"
29  }
30 ]
```

Ejercicio3 GET /categorias/:id que devuelva la categoría con el ID especificado y además, incluya todos los productos que pertenecen a esa categoría:

```
277
app.get('/categorias/:id', async (req, res) => {
  const { id } = req.params;
  const [categoriaRows] = await db.query('SELECT * FROM categorias WHERE id = ?', [id]);
  const categoria = categoriaRows[0];
  const [productosRows] = await db.query('SELECT * FROM productos WHERE categoria_id = ?', [id]);
  res.json({...categoria, productos: productosRows});
});
```

Resultado:



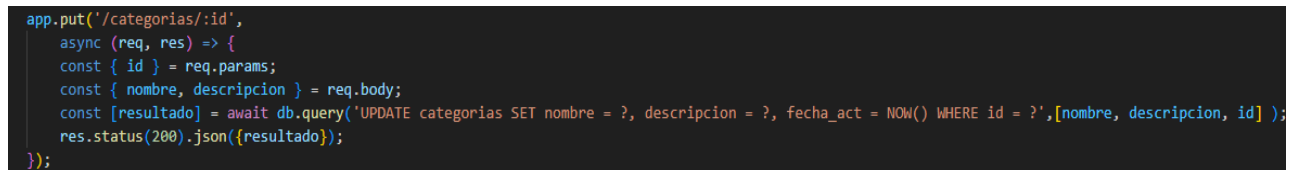
```
GET http://localhost:3001/categorias/1

Status: 200 OK Size: 481 Bytes Time: 39 ms

Response
1 {
2   "id": 1,
3   "nombre": "Electrónica",
4   "descripcion": "Dispositivos electrónicos y gadgets",
5   "fecha_alta": "2025-11-12T04:49:59.000Z",
6   "fecha_act": "2025-11-12T04:49:59.000Z",
7   "productos": [
8     {
9       "id": 1,
10      "nombre": "Laptop",
11      "precio": "1500.50",
12      "stock": 10,
13      "categoria_id": 1,
14      "fecha_alta": "2025-11-12T04:50:20.000Z",
15      "fecha_act": "2025-11-12T04:50:20.000Z"
16    },
17    {
18      "id": 2,
19      "nombre": "Mouse",
20      "precio": "25.99",
21      "stock": 50,
22      "categoria_id": 1,
23      "fecha_alta": "2025-11-12T04:50:20.000Z",
24      "fecha_act": "2025-11-12T04:50:20.000Z"
25    }
26  ]
27 }
```

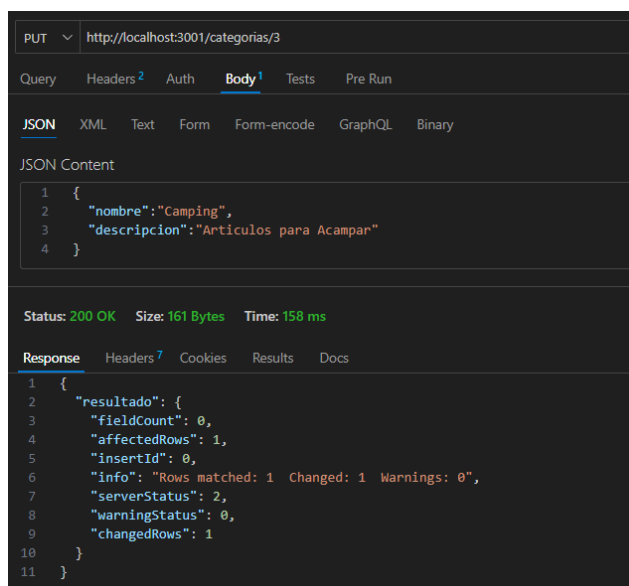
Ejercicio4 PUT /categorias/:id que permita actualizar todos los datos de la categoría con el ID :

Código:



```
app.put('/categorias/:id',
  async (req, res) => {
    const { id } = req.params;
    const { nombre, descripcion } = req.body;
    const [resultado] = await db.query('UPDATE categorias SET nombre = ?, descripcion = ?, fecha_act = NOW() WHERE id = ?', [nombre, descripcion, id ]);
    res.status(200).json({resultado});
  });
```

Resultado:

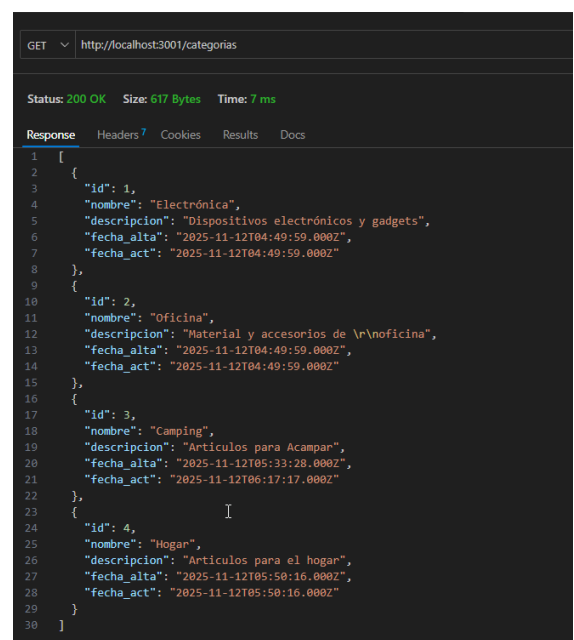


```
PUT http://localhost:3001/categorias/3

JSON Content
1 {
2   "nombre": "Camping",
3   "descripcion": "Articulos para Acampar"
4 }

Status: 200 OK Size: 161 Bytes Time: 158 ms

Response
1 {
2   "resultado": {
3     "fieldCount": 0,
4     "affectedRows": 1,
5     "insertId": 0,
6     "info": "Rows matched: 1 Changed: 1 Warnings: 0",
7     "serverStatus": 2,
8     "warningStatus": 0,
9     "changedRows": 1
10  }
11 }
```



```
GET http://localhost:3001/categorias

Status: 200 OK Size: 617 Bytes Time: 7 ms

Response
1 [
2   {
3     "id": 1,
4     "nombre": "Electrónica",
5     "descripcion": "Dispositivos electrónicos y gadgets",
6     "fecha_alta": "2025-11-12T04:49:59.000Z",
7     "fecha_act": "2025-11-12T04:49:59.000Z"
8   },
9   {
10    "id": 2,
11    "nombre": "Oficina",
12    "descripcion": "Material y accesorios de \r\noficina",
13    "fecha_alta": "2025-11-12T04:49:59.000Z",
14    "fecha_act": "2025-11-12T04:49:59.000Z"
15  },
16  {
17    "id": 3,
18    "nombre": "Camping",
19    "descripcion": "Articulos para Acampar",
20    "fecha_alta": "2025-11-12T05:33:28.000Z",
21    "fecha_act": "2025-11-12T06:17:17.000Z"
22  },
23  {
24    "id": 4,
25    "nombre": "Hogar",
26    "descripcion": "Articulos para el hogar",
27    "fecha_alta": "2025-11-12T05:50:16.000Z",
28    "fecha_act": "2025-11-12T05:50:16.000Z"
29  }
30 ]
```

Ejercicio5 DELETE /categorias/:id que elimine la categoría indicada:

Código;

```
app.delete('/categorias/:id',
  async (req, res) => {
    const { id } = req.params;
    const resultado = await db.query('DELETE FROM categorias WHERE id = ?', [id]);
    res.json({
      mensaje: 'Categoría y productos asociados eliminados correctamente',
    });
  });
```

Resultado:

DELETE http://localhost:3001/categorias/1

Query Headers 2 Auth **Body 1** Tests Pre Run

Status: 200 OK Size: 71 Bytes Time: 81 ms

Response Headers 7 Cookies Results Docs

```
1 {
2   "mensaje": "Categoría y productos asociados eliminados correctamente"
3 }
```

	id	nombre	descripcion	fecha_alta	fecha_act
☐	2	Oficina	Material y accesorios de oficina	2025-11-11 22:49:59	2025-11-11 22:49:59
☐	3	Camping	Articulos para Acampar	2025-11-11 23:33:28	2025-11-12 00:17:17
☐	4	Hogar	Articulos para el hogar	2025-11-11 23:50:16	2025-11-11 23:50:16

Opciones extra

	id	nombre	precio	stock	categoria_id	fecha_alta	fecha_act
☐	3	Cuaderno	5.00	100	2	2025-11-11 22:50:20	2025-11-11 22:50:20
☐	4	Lapicero	1.50	200	2	2025-11-11 22:50:20	2025-11-11 22:50:20

Ejercicio6 POST /productos que permita registrar un nuevo producto:

Código:

```
app.post('/productos',
  async (req, res) => {
    const { nombre, precio, stock, categoria_id } = req.body;
    const resultado = await db.query(
      `INSERT INTO productos (nombre, precio, stock, categoria_id)
      VALUES (?, ?, ?, ?)`,
      [nombre, precio, stock, categoria_id]
    );
    res.status(201).json({resultado});
  });
```

Resultado:

POST http://localhost:3001/productos

Query Headers 2 Auth **Body 1** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "nombre": "Carpa",
3   "precio": 200.50,
4   "stock": 10,
5   "categoria_id": 3
6 }
```

Status: 201 Created Size: 128 Bytes Time: 40 ms

Response Headers 6 Cookies Results Docs

```
1 {
2   "resultado": [
3     {
4       "fieldCount": 0,
5       "affectedRows": 1,
6       "insertId": 5,
7       "info": "",
8       "serverStatus": 2,
9       "warningStatus": 0,
10      "changedRows": 0
11     },
12     null
13   ]
14 }
```

Ejercicio7 GET /productos que devuelva todos los productos y muestre el nombre de la categoría a la que pertenece cada uno:

Código:

```
app.get('/productos',
  async (req, res) => {
    const [rows] = await db.query(`
      SELECT p.id,
      p.nombre,
      p.precio,
      p.stock,
      p.categoria_id,
      c.nombre AS categoria_nombre,
      p.fecha_alta,
      p.fecha_act
      FROM productos p
      INNER JOIN categorias c ON p.categoria_id = c.id
      ORDER BY p.id ASC
    `);res.json(rows);
  });
```

Resultado:

GET http://localhost:3001/productos

Status: 200 OK Size: 545 Bytes Time: 45 ms

Response

```
{
  "id": 1,
  "nombre": "Carpa",
  "precio": 200.50,
  "stock": 10,
  "categoria_id": 3,
  "categoria_nombre": "Camping",
  "fecha_alta": "2025-11-12T06:40:32.000Z",
  "fecha_act": "2025-11-12T06:40:32.000Z"
},
{
  "id": 2,
  "nombre": "Lapicero",
  "precio": 1.50,
  "stock": 200,
  "categoria_id": 2,
  "categoria_nombre": "Oficina",
  "fecha_alta": "2025-11-12T04:50:20.000Z",
  "fecha_act": "2025-11-12T04:50:20.000Z"
},
{
  "id": 3,
  "nombre": "Carpa",
  "precio": 200.50,
  "stock": 10,
  "categoria_id": 3,
  "categoria_nombre": "Camping",
  "fecha_alta": "2025-11-12T06:40:32.000Z",
  "fecha_act": "2025-11-12T06:40:32.000Z"
}
```

Ejercicio8 GET /productos/:id que devuelva la información de un producto por su ID:

Código:

```
app.get('/productos/:id',
  async (req, res) => {
    const { id } = req.params;
    const [rows] = await db.query(`
      SELECT p.id,
      p.nombre,
      p.precio,
      p.stock,
      p.categoria_id,
      c.nombre AS categoria_nombre,
      p.fecha_alta,
      p.fecha_act
      FROM productos p
      INNER JOIN categorias c ON p.categoria_id = c.id
      WHERE p.id = ?
    `, [id]);
    res.json(rows[0]);
  });
```

Resultado:

GET http://localhost:3001/productos/5

Status: 200 OK Size: 179 Bytes Time: 4 ms

Response

```
{
  "id": 5,
  "nombre": "Carpa",
  "precio": 200.50,
  "stock": 10,
  "categoria_id": 3,
  "categoria_nombre": "Camping",
  "fecha_alta": "2025-11-12T06:40:32.000Z",
  "fecha_act": "2025-11-12T06:40:32.000Z"
}
```

Ejercicio9 PUT /productos/:id que permita actualizar todos los datos de un producto, incluyendo la opción de cambiar la categoría a la que pertenece mediante categoria_id:

Código:

```
app.put('/productos/:id',  
  async (req, res) => {  
    const { id } = req.params;  
    const { nombre, precio, stock, categoria_id } = req.body  
    const resultado = await db.query(  
      `UPDATE productos SET nombre = ?, precio = ?, stock = ?, categoria_id = ?, fecha_act = NOW()  
      WHERE id = ?`,  
      [nombre, precio, stock, categoria_id, id]);  
    res.status(200).json(resultado);  
  });
```

Resultado:

PUT http://localhost:3001/productos/5

JSON Content

```
1 {  
2   "nombre": "linterna",  
3   "precio": 50.00,  
4   "stock": 70,  
5   "categoria_id": 3  
6 }
```

Status: 200 OK Size: 154 Bytes Time: 34 ms

Response

```
1 [  
2   {  
3     "fieldCount": 0,  
4     "affectedRows": 1,  
5     "insertId": 0,  
6     "info": "Rows matched: 1 Changed: 1 Warnings: 0",  
7     "serverStatus": 2,  
8     "warningStatus": 0,  
9     "changedRows": 1  
10  },  
11 null
```

GET http://localhost:3001/productos/5

JSON Content

```
1 {  
2   "nombre": "linterna",  
3   "precio": 50.00,  
4   "stock": 70,  
5   "categoria_id": 3  
6 }
```

Status: 200 OK Size: 181 Bytes Time: 3 ms

Response

```
1 {  
2   "id": 5,  
3   "nombre": "linterna",  
4   "precio": "50.00",  
5   "stock": 70,  
6   "categoria_id": 3,  
7   "categoria_nombre": "Camping",  
8   "fecha_alta": "2025-11-12T06:40:32.000Z",  
9   "fecha_act": "2025-11-12T06:58:10.000Z"  
10 }
```

Ejercicio10 PATCH /productos/:id/stock que permita incrementar o decrementar el stock de un producto enviando al body la cantidad que se desea sumar o restar:

Código:

```
app.patch('/productos/:id/stock',  
  async (req, res) => {  
    const { id } = req.params;  
    const { cantidad } = req.body;  
    const [productoRows] = await db.query('SELECT * FROM productos WHERE id = ?', [id]);  
    const producto = productoRows[0];  
    const nuevoStock = producto.stock + cantidad;  
    if (nuevoStock < 0) {  
      return res.status(400).json({ mensaje: 'El stock no puede ser negativo' });  
    }  
    await db.query(  
      'UPDATE productos SET stock = ?, fecha_act = NOW() WHERE id = ?',  
      [nuevoStock, id] );  
    res.json({mensaje: 'Stock actualizado correctamente', });  
  });
```

Resultado:

REST client interface showing a PATCH request to `http://localhost:3001/productos/5/stock`. The request body is a JSON object: `{ "cantidad": 10.00 }`. The response status is **200 OK**, with a size of **45 Bytes** and a time of **92 ms**. The response body is a JSON object: `{ "mensaje": "Stock actualizado correctamente" }`.

REST client interface showing a GET request to `http://localhost:3001/productos/5`. The request body is a JSON object: `{ "cantidad": 10.00 }`. The response status is **200 OK**, with a size of **181 Bytes** and a time of **6 ms**. The response body is a JSON object: `{ "id": 5, "nombre": "linterna", "precio": "50.00", "stock": 80, "categoria_id": 3, "categoria_nombre": "Camping", "fecha_alta": "2025-11-12T06:40:32.000Z", "fecha_act": "2025-11-12T07:06:13.000Z" }`.